

FULL PROJECT DOCUMENTATION

A. Documentation Intent

This document describes an undergraduate research prototype centered on **structured tool representations** and **tool-supervised trajectory learning**.

The core contribution is data-centric:

- reducing tool-semantic ambiguity,
- improving multi-turn tool-call consistency,
- extracting and preserving high-value behavioral signals through trajectory structuring.

Source basis:

- Code: `gen12.py`, `train.py`, `agentTester.py`
- Artifacts: `output1.json`, `outputF.json`
- Notes: `doc.txt`, `milestone1.txt`, `extra info.txt`
- Domain framing: `domain\_research.pdf`
- Dependencies: `requirements.txt`
- Empty placeholders: `agentTester.py`, `gen12.py`, `train.py`

No claims are made beyond these files.

B. Project Overview

B.1 Project Title

Structured Tool Representation and Trajectory Supervision for Multi-Turn Tool-Use Learning (Prototype)

B.2 Objective

Investigate whether **normalized, compressed, and structured tool trajectories** improve learned tool-use consistency in a smaller model (3B class) under supervised fine-tuning.

B.3 Problem Statement

Raw and synthetic tool-use logs often contain:

1. inconsistent tool naming,
2. noisy or redundant intermediate outputs,
3. weak continuity in iterative tool-use patterns,
4. lower-quality supervision signals for smaller models.

This weakens representation quality and increases tool-call entropy during learning.

B.4 Motivation (Domain-Analysis Aligned)

The domain analysis indicates that representation quality is central:

1. normalization reduces semantic fragmentation,
2. compression can preserve key behavioral signal while reducing noise,
3. structured supervision can improve consistency more effectively than raw scale alone.

B.5 What the Project Actually Delivers

1. A preprocessing system for constructing structured tool trajectories from noisy logs.
2. A QLoRA training pipeline for a small model using these structured trajectories.

3. A JSON-output validation probe to examine structural consistency of generated tool-use sequences.
4. Valid/invalid output artifacts for qualitative analysis.

C. High-Level Architecture

C.1 Architecture Summary

```
```text
[Local SWE-chat style logs: ./swe_chat_local]
 |
 v
[gen12.py] -> normalize tools/paths, clean, compress, filter, score
 |
 v
[dataset_v8.jsonl]
 |
 v
[train.py] -> QLoRA supervised fine-tuning
 |
 v
[./agent_lora_adapter]
 |
 v
[agentTester.py] -> structured JSON generation + extraction check
 |
 v
[output.json if parse succeeds]
```
```

C.2 Component Interaction

1. `gen12.py` builds `dataset\_v8.jsonl` (structured supervision dataset).
2. `train.py` learns tool-use priors from `dataset\_v8.jsonl` and saves adapter weights.
3. `agentTester.py` probes structured output consistency with strict JSON prompting and parser validation.
4. `output1.json` and `outputF.json` act as evaluation artifacts, not product outputs.

D. File-by-File Breakdown

D.1 `gen12.py`

Purpose:

- Convert raw interaction logs into high-signal structured tool trajectories.

Main logic:

1. Load dataset from `./swe\_chat\_local`.
2. Group by `session\_id`, order by `turn\_number`.
3. Extract `tool\_use` rows and pair with nearest `tool\_result`.
4. Map raw tool names into canonical tool set.

5. Normalize paths and clean low-value output text.
6. Compress output representation for edit/command steps.
7. Deduplicate repeated reads and repeated run commands.
8. Score/filter sessions for supervision quality.
9. Reconstruct compact `goal` and `meta`.
10. Write JSONL dataset.

Inputs:

- Dataset rows containing fields such as `session\_id`, `turn\_number`, `role`, `tool\_name`, `tool\_input\_json`, `content`.

Outputs:

- `dataset\_v8.jsonl`

Role:

- Primary representation-learning data pipeline.

D.2 `train.py`

Purpose:

- Fine-tune a smaller model on structured tool-use supervision.

Main logic:

1. Load `Qwen/Qwen2.5-Coder-3B-Instruct`.
2. Apply 4-bit quantization setup.
3. Prepare model for k-bit fine-tuning and apply LoRA.
4. Read/validate structured JSONL samples.
5. Format each sample into chat-style supervised text.
6. Train with TRL `SFTTrainer`.
7. Save adapter/tokenizer to `./agent\_lora`.

Inputs:

- `dataset\_v8.jsonl`, base model weights.

Outputs:

- Adapter and tokenizer artifacts in `./agent\_lora`.

Role:

- Learns structured invocation patterns and tool-semantics priors from curated trajectories.

D.3 `agentTester.py`

Purpose:

- Validate whether the trained setup can produce structurally coherent JSON tool trajectories.

Main logic:

1. Load tokenizer + quantized base model.
2. Optionally load LoRA adapter (`USE\_ADAPTER` flag).
3. Prompt model with strict schema constraints.
4. Generate response.
5. Extract first balanced JSON object.
6. Save `output.json` on successful parse.

Inputs:

- Task prompt string + model/adapters.

Outputs:

- `output.json` if valid parse succeeds.

Role:

- Structured output probe for representation quality.

D.4 `output1.json`

- Valid structured output sample.
- Demonstrates schema-conformant tool-use sequence generation.

D.5 `outputF.json`

- Invalid/truncated sample.
- Demonstrates residual inconsistency and parse failure mode.

D.6 `doc.txt`

- Detailed project rationale and research interpretation around structured tool-use learning.

D.7 `milestone1.txt`

- Milestone-focused narrative emphasizing data preparation and trajectory quality.

D.8 `domain\_research.pdf`

- Domain framing: normalization, trajectory quality, and tool semantic consistency as central research levers.

D.9 `extra info.txt`

- Team contribution mapping and collaboration context.

D.10 `requirements.txt`

- Dependency list used in setup.

D.11 `agentTester.py`, `gen12.py`, `train.py`

- Empty placeholder files.

E. Technical Implementation Deep Dive

E.1 Canonical Tool Representation

`TOOL\_MAP` consolidates varied raw tool labels into a fixed schema:

- `read\_files`
- `search\_files`
- `modify\_files`
- `write\_files`
- `run\_command`

Research relevance:

- reduces tool entropy,
- improves semantic consistency of supervision tokens,
- narrows action-space fragmentation.

E.2 Normalization and Cleaning

Implemented in preprocessing:

1. Path normalization (`/Users/...`, `/home/...` -> `/workspace/repo` style patterns).
2. Removal of reminder/log noise.
3. Cleanup of low-value outputs.
4. Line-aware truncation of verbose tool results.

Research relevance:

- increases signal density,
- lowers irrelevant lexical variance,
- supports cleaner representation learning.

E.3 Compression Strategy

Implemented behaviors:

1. Edit/write outputs are canonicalized (`Modified file: ...`).
2. Command outputs are reduced to concise success/failure signals where possible.
3. Long outputs are trimmed while preserving failure indicators.

Research tradeoff:

- improves token efficiency and structural regularity,
- may lose fine-grained patch semantics.

E.4 Session Filtering and Scoring

Key gates in `process\_session`:

1. minimum step threshold,
2. mandatory modification/write presence,
3. dedupe and trim passes,
4. score threshold based on tool diversity and balance.

Scoring signals include:

- presence of `read\_files`, `modify\_files`, `search\_files`, `run\_command`,
- diversity bonus,
- penalty for run-command dominance.

Research relevance:

- prioritizes high-quality structured supervision over raw volume.

E.5 Goal and Meta Reconstruction

`build\_goal` infers compact objective labels from command/output cues.

`build\_meta` summarizes:

- tool count,
- verification presence,
- multi-file involvement.

Research relevance:

- creates lightweight supervisory context for trajectory-level structure.

E.6 Training Method (`train.py`)

Model and method:

- Base: `Qwen/Qwen2.5-Coder-3B-Instruct`
- QLoRA with 4-bit quantization (`nf4`)
- LoRA target modules:
 - `q\_proj`, `k\_proj`, `v\_proj`, `o\_proj`,
 - `gate\_proj`, `up\_proj`, `down\_proj`

Training config (as implemented):

- epochs: `3`
- learning rate: `2e-4`
- batch size: `1`
- gradient accumulation: `16`
- max sequence length: `2048`
- LoRA rank/alpha/dropout: `32/64/0.05`

Research relevance:

- tests whether smaller models can absorb structured tool-use priors from curated trajectories.

E.7 Inference Validation (`agentTester.py`)

Mechanism:

1. strict schema-focused prompt,
2. sampled generation,
3. robust balanced-brace extraction.

Research relevance:

- evaluates practical structural consistency of generated tool trajectories.

F. Domain Analysis Integration

From `domain\_research.pdf`, core aligned ideas:

1. format compliance alone is insufficient without tool-semantic consistency,
2. synthetic trajectories may miss realistic iterative structure,
3. real logs are valuable but noisy,
4. preprocessing quality is central to representation quality.

Repository alignment:

1. `gen12.py` directly operationalizes normalization, compression, and filtering.
2. `train.py` applies structured supervision to a smaller model.
3. `agentTester.py` checks whether structured priors appear in outputs.

G. Installation and Setup

G.1 Prerequisites

1. Python 3.10+
2. GPU environment recommended for 3B QLoRA workflow

3. Access to Hugging Face model downloads

G.2 Dependencies

From `requirements.txt`:

- torch
- transformers
- datasets
- peft
- trl
- bitsandbytes
- tqdm
- accelerate

G.3 Setup Commands

```
```powershell
python -m venv .venv
.\.venv\Scripts\Activate.ps1
pip install -r requirements.txt
```
```

H. Execution Guide

H.1 Script Order

```
```powershell
python "gen12.py"
python "train.py"
python "agentTester.py"
```
```

H.2 Expected Behavior

1. Preprocessing: writes `dataset\_v8.jsonl` and reports kept/dropped sessions.
2. Training: runs QLoRA SFT and saves adapter/tokenizer under `./agent\_lora`.
3. Inference probe: writes `output.json` on successful parse, otherwise reports invalid/truncated output.

H.3 Common Operational Issues

1. Missing `./swe\_chat\_local` path.
2. Resource constraints during model load/training.
3. Parse failures from truncated or malformed generated output.

I. Inputs and Outputs

I.1 Preprocessing Input

Structured session logs from local disk dataset export.

I.2 Dataset Output Schema

```
```json
{
 "goal": "...",
 "meta": {
 "tool_count": 0,
 "has_verification": true,
 "multi_file": false
 },
 "m": [
 {"t": "run_command", "a": {}, "o": "..."}
],
 "end": 1
}
```
```

I.3 Output Artifacts for Evaluation

- `output1.json`: valid structured sample.
- `outputF.json`: invalid/truncated sample.

These artifacts are used as consistency probes, not standalone performance metrics.

J. Results and Observations (Repository-Evidence Bound)

Current repository evidence supports:

1. functional structured-trajectory preprocessing and training pipeline,
2. successful structured JSON generation in some runs,
3. remaining reliability gaps in generation consistency.

No benchmark tables or formal metric dashboards are included in the files.

K. Limitations (Research-Appropriate)

1. Evaluation in repository is mostly qualitative/structural.
2. No integrated large-scale metric harness is provided.
3. Compression may remove detailed edit semantics.
4. Post-generation validation is parser-based, not full strict schema enforcement.

These are normal constraints for an undergraduate research prototype and point to clear next steps.

L. Future Scope

1. Add strict schema validation + constrained decoding/retry loops.
2. Add quantitative consistency metrics (e.g., validity and tool-sequence fidelity).
3. Improve compressed edit representation to retain more semantic detail.
4. Expand trajectory difficulty and long-horizon tool-use sequences.
5. Add experiment tracking and reproducibility controls.

M. Code Quality Review (Balanced)

Strengths:

1. clear modular preprocessing design,
2. coherent data-centric supervision strategy,
3. reproducible script order from data processing to output probing.

Improvement areas:

1. externalize configs for easier experimentation,
2. add targeted tests for preprocessing and parsing components,
3. add formal evaluation scripts to strengthen empirical reporting.

N. ASCII Diagrams

N.1 Workflow

```text

Raw logs

- > normalization + compression + filtering
- > structured JSONL supervision
- > QLoRA fine-tuning
- > structured JSON output probe

```

N.2 Module Interaction

```text

gen12.py -> dataset\_v8.jsonl -> train.py -> agent\_lora  
                                  -> agentTester.py

```

N.3 Execution Flow

```text

Start

- > preprocess trajectories
- > train tool-supervised model
- > run structured output validation
- > compare valid/invalid artifacts

End

```

N.4 Data Flow

```text

session rows -> extracted steps -> normalized/compressed steps -> filtered trajectory -> JSONL sample

```

O. Contribution Note (from `extra info.txt`)

- Research and dataset pattern: Suhas, Swayam, Nikhil
- Dataset filtering/training/formatting collaboration: Shrikar, Dalip
- Tool-interaction pipeline implementation: Harshit, Rahul

P. Final Framing Statement

This project is best understood as ****data-centric research on structured tool embeddings and trajectory representations****.

Its primary contribution is improved supervision quality for learning consistent multi-turn tool-use patterns in smaller models; generated coding-task JSON outputs are supporting artifacts for that research objective.